PYTHON PROGRAMMING ACTIVITY-BASED-LEARNING ASSIGNMENT REPORT

10 Python Case Studies

Student detail	Information
Name	E. Blessy Johns
Roll number	26000019
Branch	CSE
Section	01

CONTENTS

1 ***Case 4 – Employee Payroll and Deduction Analysis***

2 ***Case 8 – Manufacturing Cost and Profitability Analysis***

3 ***Case 10 – Campus Event Financial Planning***

4 ***Case 11 – Examination Result and Grace-Mark Validation***

5 ***Case 12 – Attendance Recovery Computation***

6 ***Case 16 – Multi-Day Water Tank Simulation***

7 ***Case 18 – Fuel Consumption and Route Comparison***

8 ***Case 51 – Bus Passenger Occupancy Simulation***

9 ***Case 53 – Food Waste Monitoring System***

10 ***Case 63 – Multi-Product Price Fluctuation Analysis***

Case 4: Employee Payroll and Deduction Analysis

Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

Objective

To calculate the gross salary, deduction, and net salary based on the basic salary, allowance, overtime, and incentive.

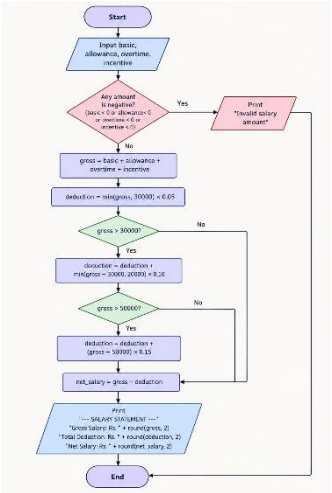
Assumptions

1. All salary amounts are entered as non-negative numbers.
2. Gross salary is the sum of basic salary, allowances, overtime, and incentives.
3. Progressive deductions are calculated as follows:
    - 5% on the first ₹30,000.
    - 10% on the portion between ₹30,000 and ₹50,000.
    - 15% on the portion above ₹50,000.
4. Net salary is calculated by subtracting the total deduction from gross salary.
5. The deduction slabs are assumed for this assignment.

Algorithm

1. Start.
2. Input basic salary, allowance, overtime, and incentive.
3. Calculate gross salary by adding all four amounts.
4. Calculate 5% deduction on the first ₹30,000.
5. If gross salary exceeds ₹30,000, calculate 10% deduction on the portion up to ₹50,000.
6. If gross salary exceeds ₹50,000, calculate 15% deduction on the amount above ₹50,000.
7. Add all deductions.
8. Calculate net salary by subtracting the deduction from gross salary.
9. Display a formatted salary statement showing gross salary, deduction, and net salary.
10. Stop.

Flowchart



Pseudocode

```
START
INPUT basic, allowance, overtime, incentive
gross ← basic + allowance + overtime + incentive

deduction ← MIN(gross, 30000) × 0.05

IF gross > 30000 THEN
    deduction ← deduction + MIN(gross - 30000, 20000) × 0.10
END IF

IF gross > 50000 THEN
    deduction ← deduction + (gross - 50000) × 0.15
END IF

net_salary ← gross - deduction

OUTPUT gross, deduction, net_salary
STOP
```

Python Source Code

```
basic = float(input("Enter basic salary: "))
allowance = float(input("Enter allowance: "))
overtime = float(input("Enter overtime amount: "))
incentive = float(input("Enter incentive: "))

if basic < 0 or allowance < 0 or overtime < 0 or incentive < 0:
    print("Invalid salary amount")
else:
    gross = basic + allowance + overtime + incentive

    deduction = min(gross, 30000) * 0.05

    if gross > 30000:
        deduction = deduction + min(gross - 30000, 20000) * 0.10

    if gross > 50000:
        deduction = deduction + (gross - 50000) * 0.15

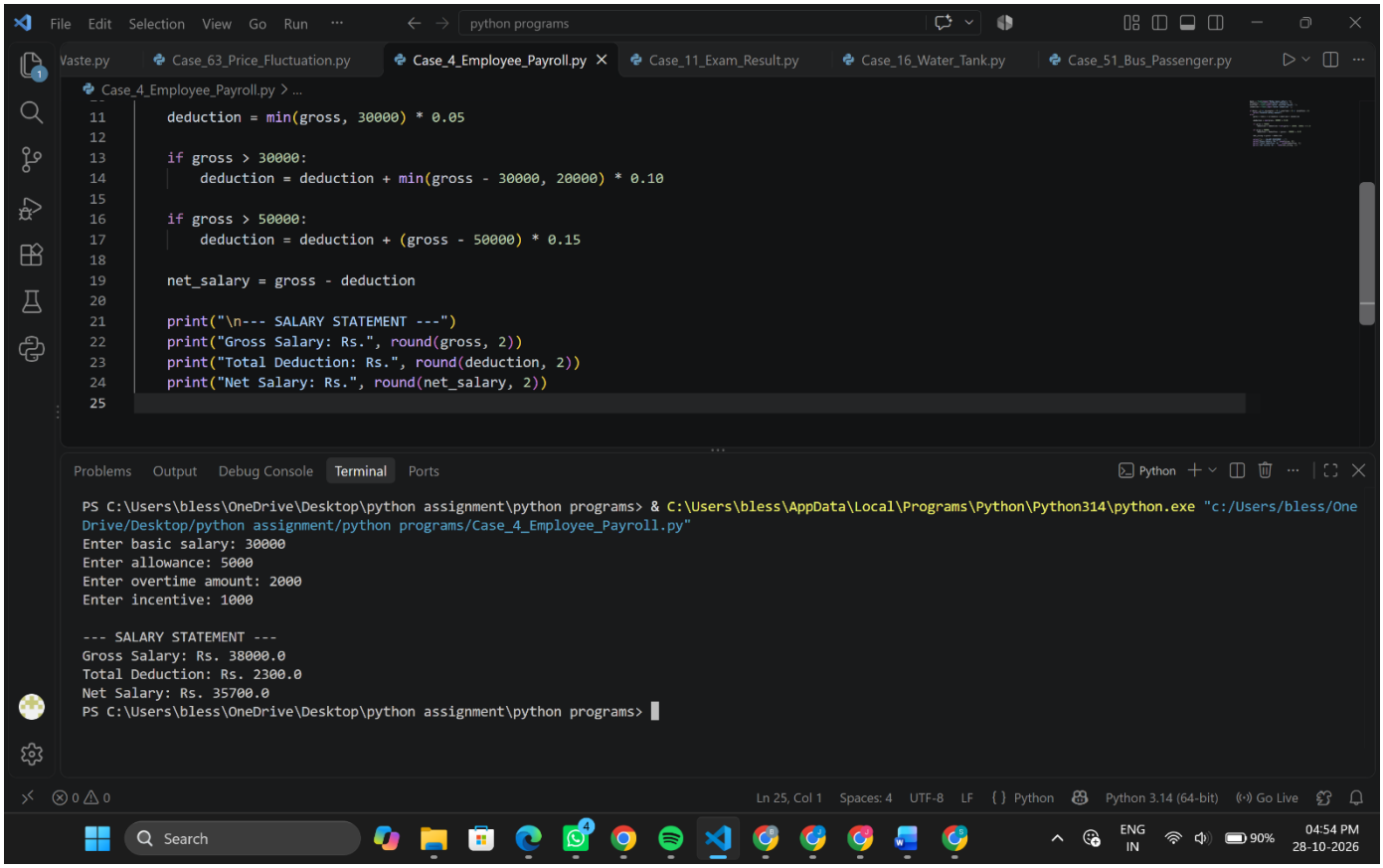
    net_salary = gross - deduction

    print("\n--- SALARY STATEMENT ---")
    print("Gross Salary: Rs.", round(gross, 2))
    print("Total Deduction: Rs.", round(deduction, 2))
   print("Net Salary: Rs.", round(net_salary, 2))
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Basic 30000, allowance 5000, overtime 2000, incentive 1000	Gross=38000; deduction=2300; net=35700	Pass if values match – Pass
TC2	All components = 0	Gross=0; deduction=0; net=0	Pass if values match – Pass
TC3	Basic 60000, other components 0	Gross=60000; deduction=5000; net=55000	Pass if values match – Pass
TC4	Any salary component is negative	Invalid salary amount	Pass if values match – Pass

Output Screenshot



Conclusion

The program calculates gross salary, applies progressive deductions to different salary slabs, and displays the total deduction and net salary in a formatted salary statement.

Case 8: Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

Objective

To calculate total cost and profit and compare profit with a target value.

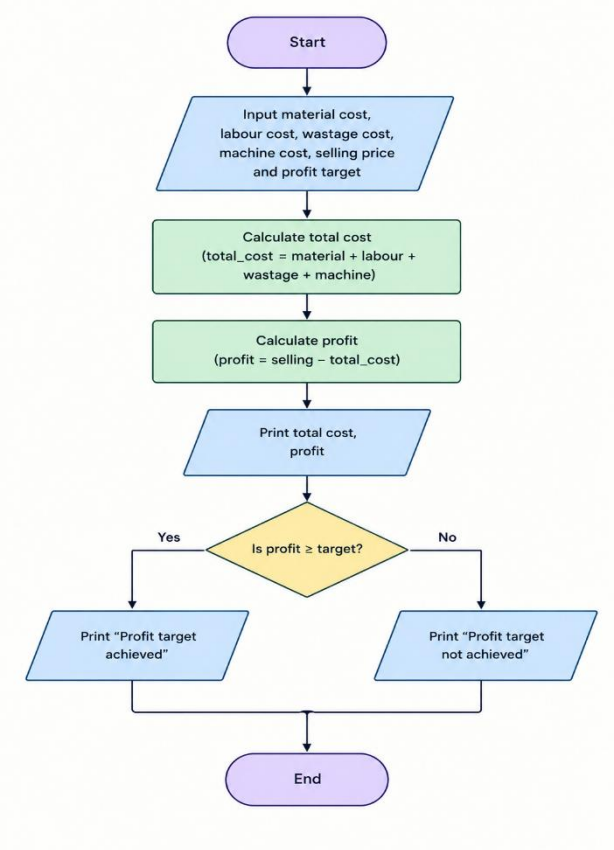
Assumptions

1. All costs, selling price, and target are numeric values.
2. Total cost is the sum of the four costs.
3. Profit equals selling price minus total cost. The selling price is treated as total revenue for the batch.

Algorithm

1. Start.
2. Read material, labour, wastage, and machine costs.
3. Read selling price and profit target.
4. Add the four costs to calculate total cost.
5. Subtract total cost from selling price to calculate profit.
6. If profit is at least the target, report target achieved; otherwise report target not achieved.
7. Display total cost and profit.
8. Stop.

Flowchart



Pseudocode

```
START
INPUT material, labour, wastage, machine
INPUT selling_price, target
total_cost ← material + labour + wastage + machine
profit ← selling_price – total_cost
OUTPUT total_cost, profit
IF profit ≥ target THEN OUTPUT "Profit target achieved"
ELSE OUTPUT "Profit target not achieved"
STOP
```

Python Source Code

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour cost: "))
wastage = float(input("Enter wastage cost: "))
machine = float(input("Enter machine cost: "))
selling = float(input("Enter selling price: "))
target = float(input("Enter profit target: "))

total_cost = material + labour + wastage + machine
profit = selling - total_cost

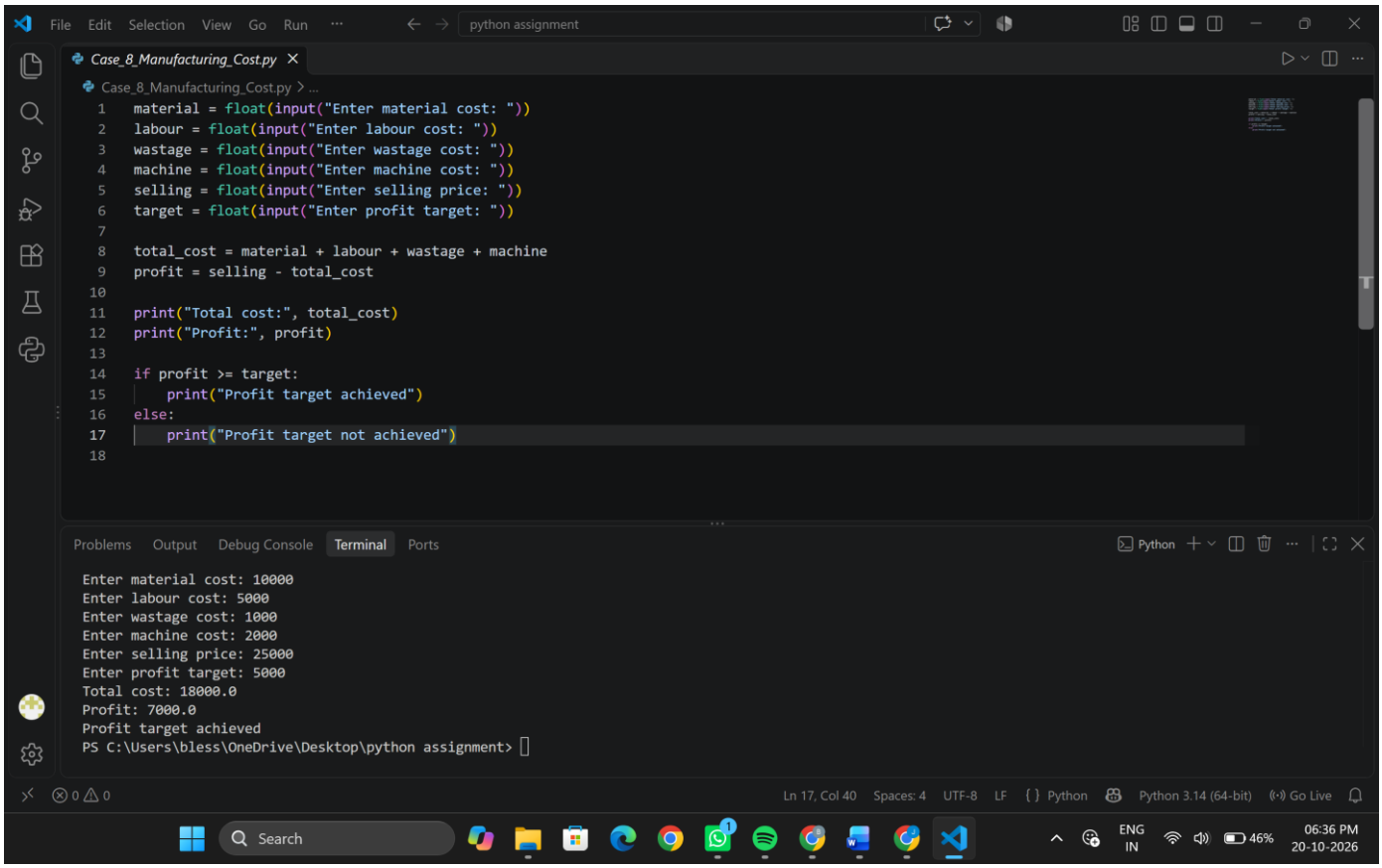
print("Total cost:", total_cost)
print("Profit:", profit)

if profit >= target:
    print("Profit target achieved")
else:
    print("Profit target not achieved")
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Costs 10000, 5000, 1000, 2000; selling=25000; target=5000	Total cost=18000; profit=7000; target achieved	Pass if values match – Pass
TC2	All costs 0; selling=0; target=0	Total cost=0; profit=0; target achieved	Pass if values match – Pass
TC3	Costs 10000, 5000, 1000, 2000; selling=19000; target=2000	Total cost=18000; profit=1000; target not achieved	Pass if values match – Pass

Output Screenshot Evidence



Conclusion

The program uses arithmetic operations and if-else selection to assess the profitability of a manufacturing batch.

Case 10: Campus Event Financial Planning

Problem Statement

Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

Objective

To support event budgeting by calculating total cost, remaining cost after sponsorship, and required registrations.

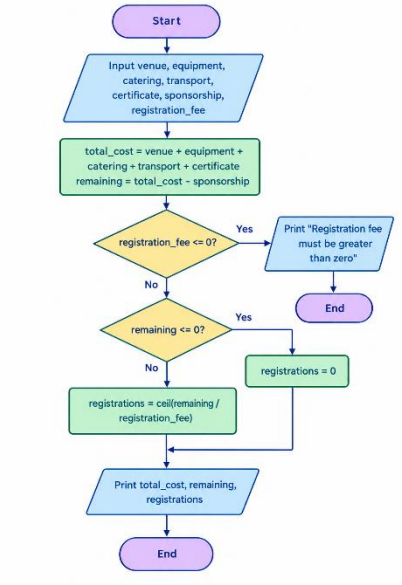
Assumptions

1. All costs, sponsorship, and registration fee are entered as numeric values.
2. Costs and sponsorship are assumed to be non-negative.
3. Registration fee must be greater than zero before registrations are calculated.
4. If sponsorship covers the full cost, no registrations are required.
5. A partial registration is impossible, so the number of registrations is rounded up when needed.

Algorithm

1. Start.
2. Read venue, equipment, catering, transport, and certificate costs.
3. Read sponsorship and registration fee.
4. Calculate total cost by adding all expenses.
5. Calculate remaining cost by subtracting sponsorship from total cost.
6. Display total cost and remaining cost.
7. If registration fee is zero or negative, display an error message.
8. Otherwise, if remaining cost is zero or less, set registrations to zero.
9. Otherwise, calculate the ceiling of remaining cost divided by registration fee.
10. Display the minimum registrations and stop.

Flowchart



Pseudocode

```
START
INPUT venue, equipment, catering, transport, certificate
INPUT sponsorship, registration_fee
total_cost ← venue + equipment + catering + transport + certificate
remaining ← total_cost – sponsorship
OUTPUT total_cost, remaining
IF registration_fee ≤ 0 THEN
OUTPUT registration fee error
ELSE IF remaining ≤ 0 THEN
registrations ← 0
ELSE
registrations ← CEILING(remaining / registration_fee)
OUTPUT registrations when registration_fee > 0
STOP
```

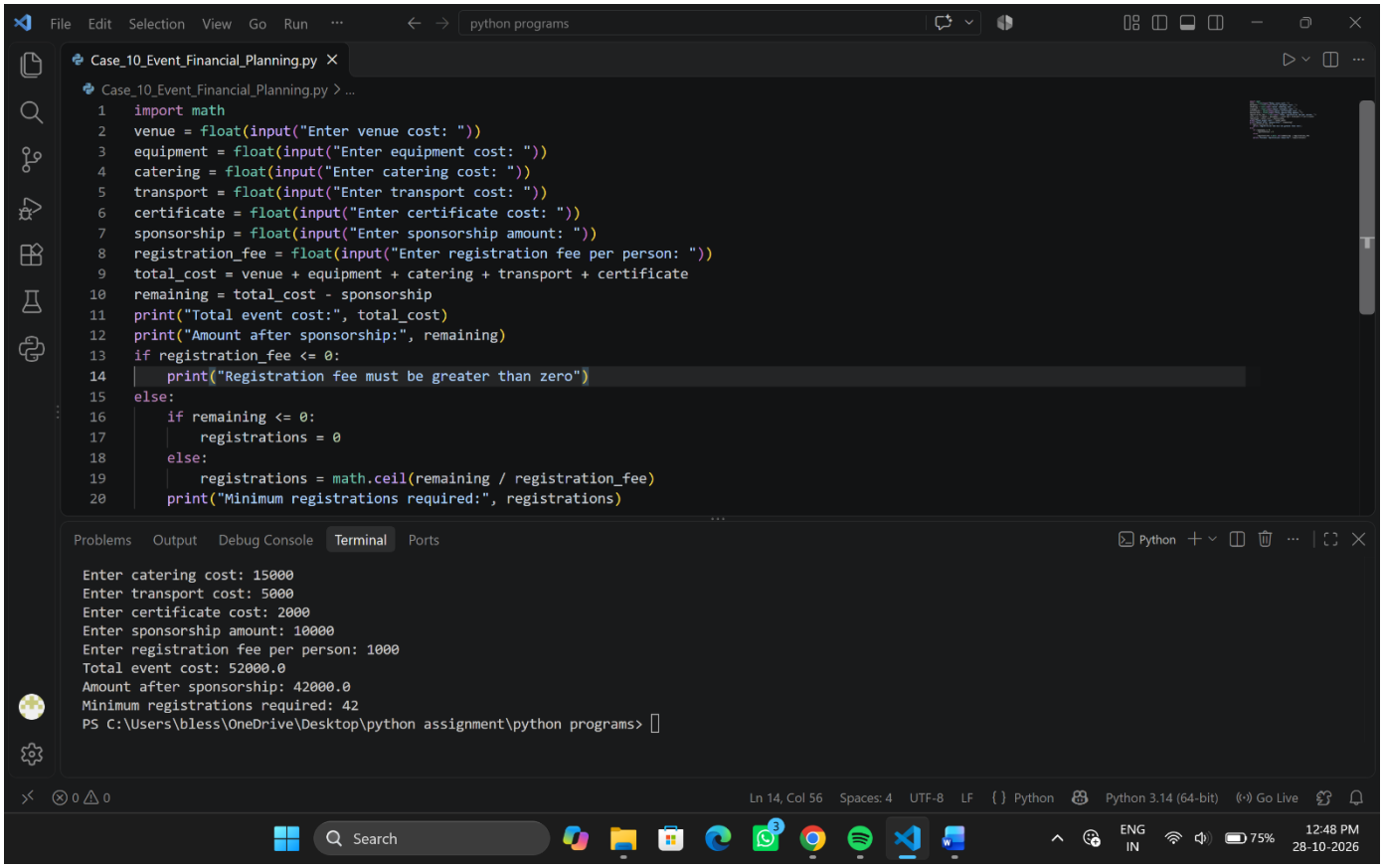
Python Source Code

```
import math
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost: "))
transport = float(input("Enter transport cost: "))
certificate = float(input("Enter certificate cost: "))
sponsorship = float(input("Enter sponsorship amount: "))
registration_fee = float(input("Enter registration fee per person: "))
total_cost = venue + equipment + catering + transport + certificate
remaining = total_cost - sponsorship
print("Total event cost:", total_cost)
print("Amount after sponsorship:", remaining)
if registration_fee <= 0:
    print("Registration fee must be greater than zero")
else:
    if remaining <= 0:
        registrations = 0
    else:
        registrations = math.ceil(remaining / registration_fee)
    print("Minimum registrations required:", registrations)
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Costs 20000,10000,15000,5000,2000; sponsorship=10000; fee=1000	Total=52000; remaining=42000; registrations=42	Pass if values match – Pass
TC2	Total cost=5000; sponsorship=5000; fee=1000	Remaining=0; registrations=0	Pass if values match – Pass
TC3	Fee=0	Validation message: registration fee must be greater than zero	Pass if message appears – Pass

Output Screenshot Evidence



Conclusion

The program demonstrates budgeting calculations, input validation, and ceiling division to estimate registrations.

Case 11: Examination Result and Grace-Mark Validation

Problem Statement

Process marks across multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

Objective

To classify marks using pass and grace thresholds and determine the overall result from the three subjects.

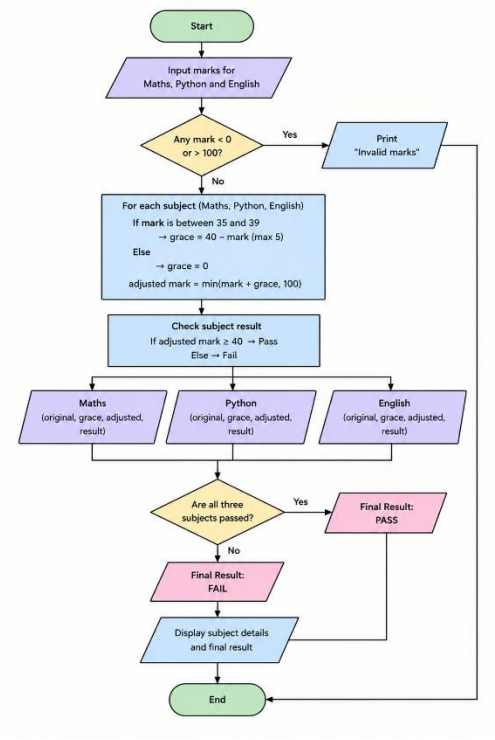
Assumptions

1. Marks are entered as integers between 0 and 100 for Maths, Python, and English.
2. A student needs 40 marks to pass each subject.
3. A subject mark from 35 to 39 can receive grace marks to reach 40.
4. A maximum of 5 grace marks can be awarded per subject.
5. Marks below 35 are treated as Fail.
6. Final marks must not exceed 100.
7. The grace-mark policy is an assumption for this assignment.

Algorithm

1. Start.
2. Input marks for Maths, Python, and English.
3. If any mark is below 0 or above 100, display Invalid marks and stop.
4. For each subject, if marks are from 35 to 39, calculate the grace marks needed to reach 40.
5. Add the grace marks to the subject marks without exceeding 100.
6. If the adjusted mark is 40 or above, classify the subject as Pass; otherwise, classify it as Fail.
7. Display original marks, grace marks awarded, adjusted marks, and subject results.
8. If all subjects are passed, display Final Result: PASS; otherwise, display Final Result: FAIL.
9. Stop.

Flowchart



Pseudocode

```
START
INPUT maths, python, english

IF any mark < 0 OR any mark > 100 THEN
    OUTPUT "Invalid marks"
ELSE
    FOR each subject mark
        grace ← 0

        IF mark >= 35 AND mark < 40 THEN
            grace ← 40 - mark
        END IF

        adjusted_mark ← MIN(mark + grace, 100)

        IF adjusted_mark >= 40 THEN
            result ← "Pass"
        ELSE
            result ← "Fail"
        END IF

        OUTPUT grace, adjusted_mark, result
    END FOR

    IF all adjusted marks >= 40 THEN
        OUTPUT "Final Result: PASS"
    ELSE
        OUTPUT "Final Result: FAIL"
    END IF
END IF
STOP
```

Python Source Code

```
maths = int(input("Enter Maths marks: "))
python = int(input("Enter Python marks: "))
english = int(input("Enter English marks: "))

if (maths < 0 or maths > 100 or
    python < 0 or python > 100 or
    english < 0 or english > 100):
    print("Invalid marks")
else:
    maths_grace = 0
    python_grace = 0
    english_grace = 0

    if 35 <= maths < 40:
        maths_grace = 40 - maths

    if 35 <= python < 40:
        python_grace = 40 - python

    if 35 <= english < 40:
```

```
    english_grace = 40 - english

maths_final = min(maths + maths_grace, 100)
python_final = min(python + python_grace, 100)
english_final = min(english + english_grace, 100)

if maths_final >= 40:
    maths_result = "Pass"
else:
    maths_result = "Fail"

if python_final >= 40:
    python_result = "Pass"
else:
    python_result = "Fail"

if english_final >= 40:
    english_result = "Pass"
else:
    english_result = "Fail"

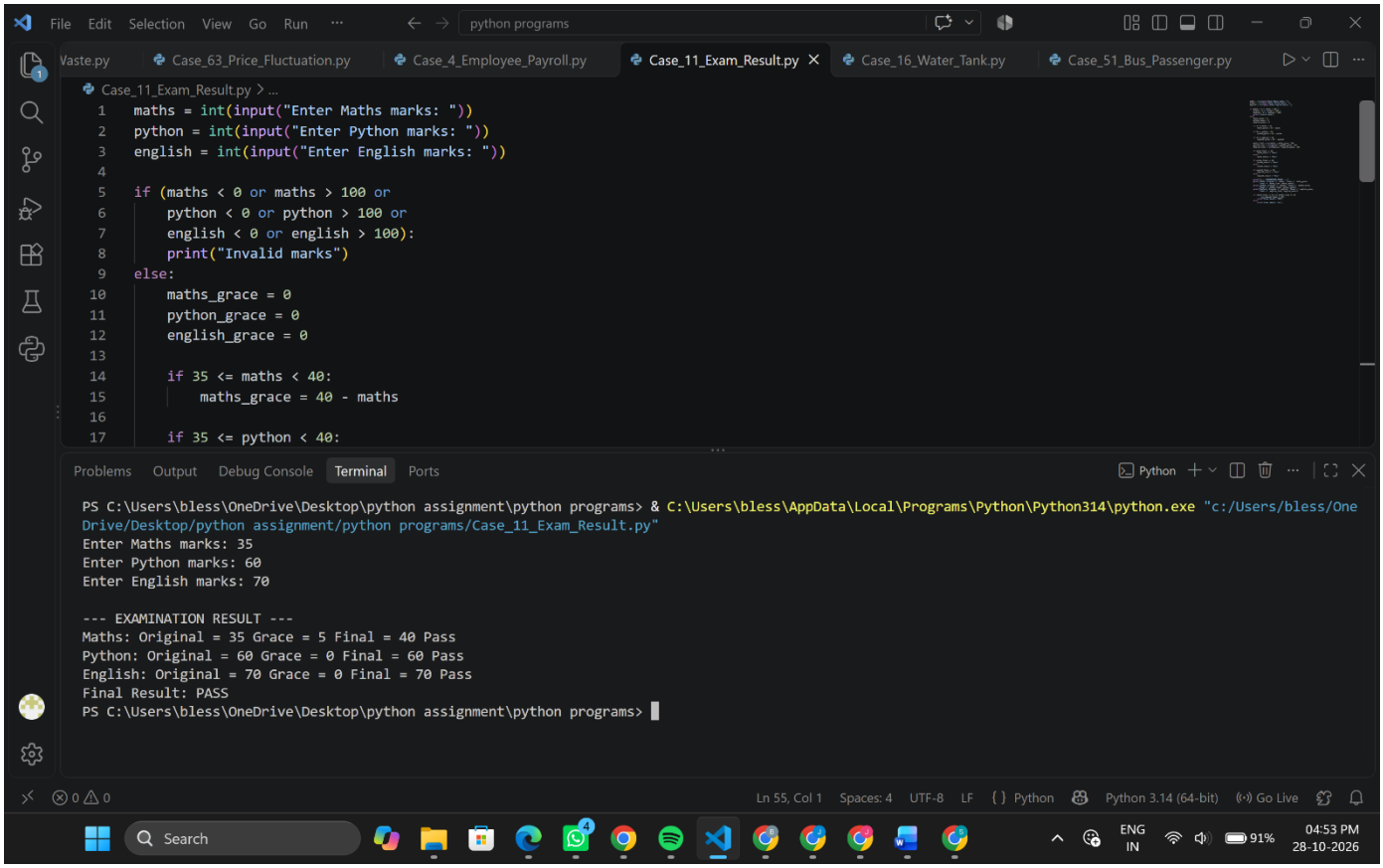
print("\n--- EXAMINATION RESULT ---")
print("Maths: Original =", maths, "Grace =", maths_grace,
      "Final =", maths_final, maths_result)
print("Python: Original =", python, "Grace =", python_grace,
      "Final =", python_final, python_result)
print("English: Original =", english, "Grace =", english_grace,
      "Final =", english_final, english_result)

if (maths_final >= 40 and python_final >= 40
        and english_final >= 40):
    print("Final Result: PASS")
else:
    print("Final Result: FAIL")
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Maths=35, Python=60, English=70	Maths: Receives 5 grace marks; adjusted marks=40,60,70; final PASS	Pass if values match – Pass
TC2	Maths=34, Python=80, English=90	Maths: Fail; Final FAIL	Pass if values match – Pass
TC3	All marks=40	No grace marks; All subjects Pass	Pass if values match – Pass
TC4	Any mark is greater than 100 or below 0	Invalid marks	Pass if same result - Pass

Output Screenshot Evidence



Conclusion

The program validates subject marks, applies the specified grace-mark policy, calculates adjusted marks without exceeding the maximum, and determines the final examination result.

Case 12: Attendance Recovery Computation

Problem Statement

Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

Objective

To calculate attendance and simulate future attendance until the required percentage is achieved.

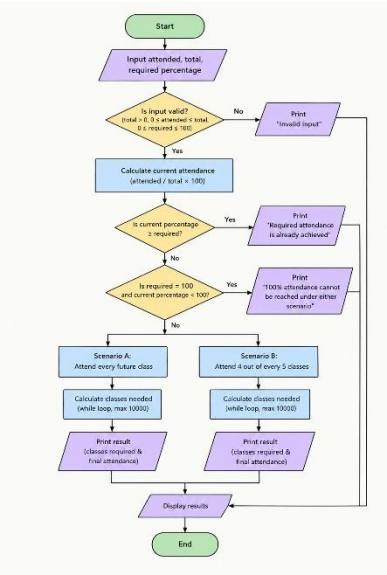
Assumptions

1. Total classes must be greater than zero.
2. Attended classes must be between zero and total classes.
3. Required attendance must be between 0% and 100%.
4. Scenario A assumes every future class is attended.
5. Scenario B assumes four classes are attended and the fifth class is missed, repeatedly.
6. The program calculates the minimum number of future classes needed to reach the required attendance under each scenario.
7. If a target cannot be reached under a scenario, the program reports that it is not achievable under that scenario.

Algorithm

1. Start.
2. Input attended classes, total classes, and required attendance percentage.
3. Validate all inputs.
4. If any input is invalid, display Invalid input and stop.
5. Calculate and display current attendance percentage.
6. For Scenario A, simulate attending every future class until the required percentage is reached, counting the number of classes needed.
7. For Scenario B, simulate a repeating pattern of four attended classes followed by one missed class.
8. After each future class, update the attended and total class counts and calculate the new percentage.
9. Stop each simulation when the required percentage is reached or the simulation limit is reached.
10. Display the number of future classes needed and the final percentage for each scenario, or report if the target was not reached.
11. Stop.

Flowchart



Pseudocode

```
START
INPUT attended, total, required

IF total <= 0 OR attended < 0 OR attended > total
   OR required < 0 OR required > 100 THEN
    OUTPUT "Invalid input"
ELSE
    current ← attended / total × 100
    OUTPUT current attendance

    FOR scenario ← A TO B
        future_attended ← attended
        future_total ← total
        classes ← 0

        WHILE future_attended / future_total × 100 < required
              AND classes < 10000
            classes ← classes + 1

            IF scenario = A THEN
                future_attended ← future_attended + 1
            ELSE IF classes MOD 5 != 0 THEN
                future_attended ← future_attended + 1
            END IF

            future_total ← future_total + 1
        END WHILE

        IF future_attended / future_total × 100 >= required THEN
            OUTPUT scenario, classes
            OUTPUT future_attended / future_total × 100
        ELSE
            OUTPUT "Target not reached in this scenario"
        END IF
    END FOR
END IF
STOP
```

Python Source Code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))
required = float(input("Enter required attendance percentage: "))

if (total <= 0 or attended < 0 or attended > total
      or required < 0 or required > 100):
   print("Invalid input")
else:
   percentage = attended / total * 100
   print("Current attendance:", round(percentage, 2), "%")

   if percentage >= required:
      print("Required attendance is already achieved")
   elif required == 100 and percentage < 100:
      print("100% attendance cannot be reached under either scenario")
```

```
else:
    # Scenario A: Attend every future class
    a_attended = attended
    a_total = total
    a_classes = 0
    while (a_attended / a_total * 100 < required) and (a_classes < 10000):
        a_attended = a_attended + 1
        a_total = a_total + 1
        a_classes = a_classes + 1

    print("\nScenario A: Attend every class")
    if a_classes >= 10000:
        print("Target not reached within limit")
    else:
        print("Classes required:", a_classes)
        print("Final attendance:", round(a_attended / a_total * 100, 2), "%")

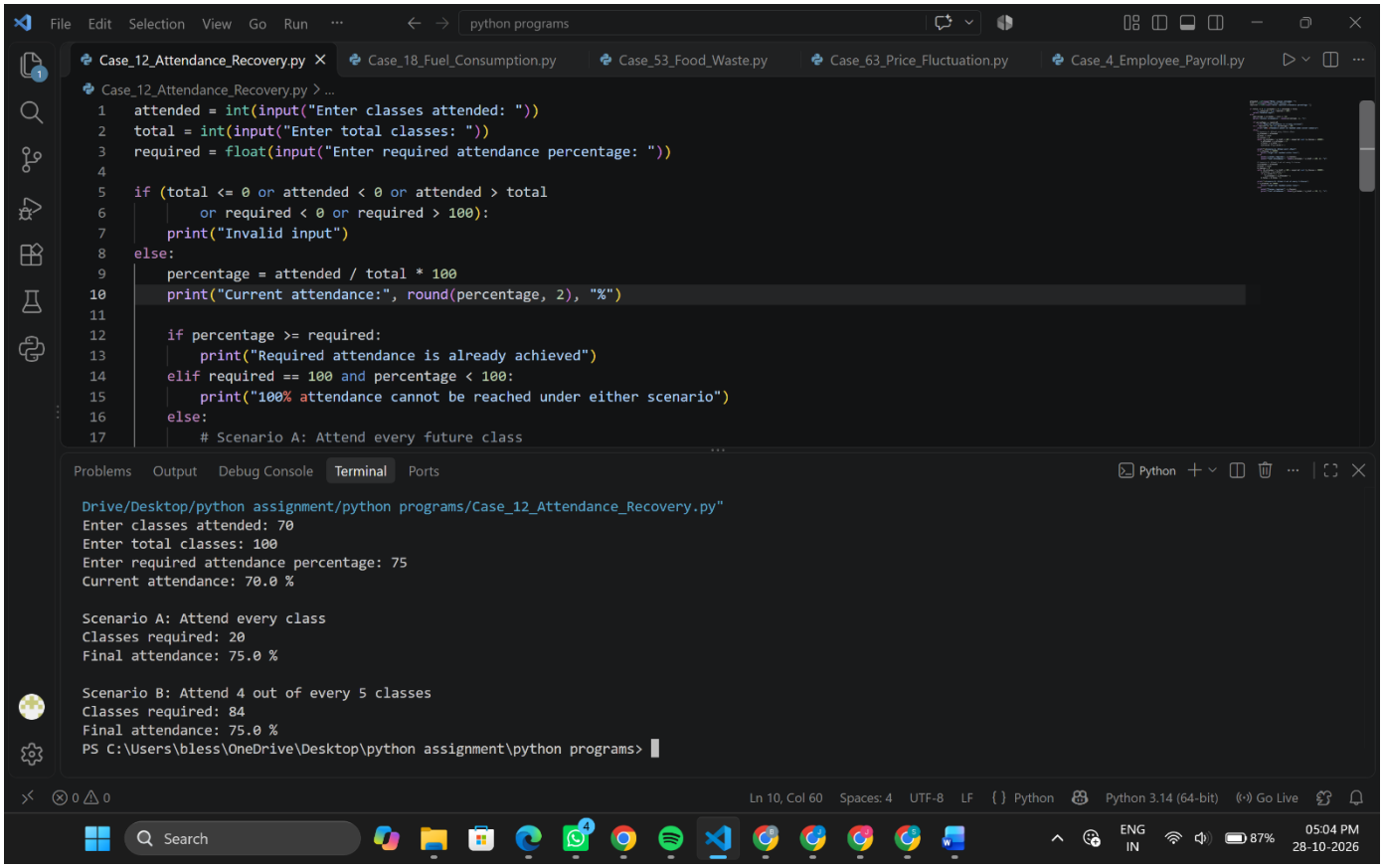
    # Scenario B: Attend 4 out of every 5 classes
    b_attended = attended
    b_total = total
    b_classes = 0
    while (b_attended / b_total * 100 < required) and (b_classes < 10000):
        b_classes = b_classes + 1
        if b_classes % 5 != 0:
            b_attended = b_attended + 1
        b_total = b_total + 1

    print("\nScenario B: Attend 4 out of every 5 classes")
    if b_classes >= 10000:
        print("Target not reached within limit")
    else:
        print("Classes required:", b_classes)
        print("Final attendance:", round(b_attended / b_total * 100, 2), "%")
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Attended=70, total=100, required=75	Scenario A: 20 classes; Scenario B: 84 classes	Pass if values match – Pass
TC2	Attended=80, total=100, required=75	Target already achieved; 0 classes needed in both scenarios	Pass if values match – Pass
TC3	Total=0	Validation error	Pass if message appears – Pass
TC4	Attended=70, total=100, required=100	Target cannot be reached by either scenario because attendance is initially below 100%	Pass if values match – Pass

Output Screenshot Evidence



Conclusion

The program demonstrates while-loop iteration and percentage updates to estimate attendance recovery.

Case 16: Multi-Day Water Tank Simulation

Problem Statement

Process changing inflow and outflow quantities, update water levels, detect overflow and shortages, and determine the first interval in which the tank reaches a specified threshold.

Objective

To model water-level changes and display the tank status each day.

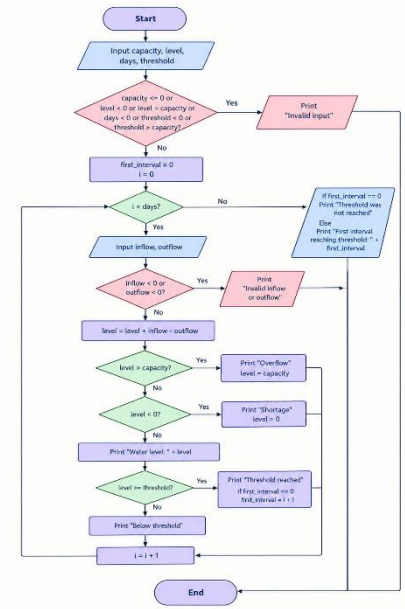
Assumptions

1. Tank capacity must be greater than zero.
2. Starting level must be between zero and capacity.
3. Number of days must be a non-negative integer.
4. Threshold must be between zero and capacity.
5. Daily inflow and outflow must be non-negative.
6. The level is capped at capacity after overflow and set to zero after shortage

Algorithm

1. Start.
2. Read tank capacity, starting water level, number of days, and threshold.
3. Validate the initial values. If invalid, display Invalid input and stop.
4. Set first interval as not found.
5. Repeat for each day: read inflow and outflow.
6. If inflow or outflow is negative, display Invalid inflow or outflow and continue to the next day.
7. Update water level = current level + inflow − outflow.
8. If water level exceeds capacity, display Overflow and set water level to capacity.
9. If water level is below zero, display Shortage and set water level to zero.
10. Display the water level.
11. If the water level reaches the threshold for the first time, record that day as the first interval.
12. Display whether the threshold has been reached.
13. After all days, display the first interval in which the threshold was reached, if any.
14. Stop.

Flowchart



Pseudocode

```
START
INPUT capacity, level, days, threshold
IF capacity ≤ 0 OR level < 0 OR level > capacity OR days < 0 OR threshold < 0 OR threshold > capacity THEN
 OUTPUT Invalid input
ELSE
 first_interval ← 0
 FOR each day
  INPUT inflow, outflow
  IF inflow < 0 OR outflow < 0 THEN
   OUTPUT Invalid inflow or outflow
  ELSE
   level ← level + inflow – outflow
   IF level > capacity THEN
    OUTPUT Overflow
    level ← capacity
   ELSE IF level < 0 THEN
    OUTPUT Shortage
    level ← 0
   OUTPUT level
   IF level ≥ threshold THEN
    OUTPUT Threshold reached
    IF first_interval = 0 THEN
     first_interval ← day number
   ELSE
    OUTPUT Below threshold
 IF first_interval = 0 THEN
  OUTPUT Threshold was not reached
 ELSE
  OUTPUT first_interval
STOP
```

Python Source Code

```
capacity = float(input("Enter tank capacity: "))
level = float(input("Enter starting water level: "))
days = int(input("Enter number of days: "))
threshold = float(input("Enter required threshold: "))

if capacity <= 0 or level < 0 or level > capacity or days < 0 or threshold < 0 or threshold > capacity:
    print("Invalid input")
else:
    first_interval = 0

    for i in range(days):
        inflow = float(input("Enter water inflow: "))
        outflow = float(input("Enter water outflow: "))

        if inflow < 0 or outflow < 0:
            print("Invalid inflow or outflow")
        else:
            level = level + inflow - outflow

            if level > capacity:
                print("Overflow")
                level = capacity
            elif level < 0:
                print("Shortage")
```

```
                level = 0

            print("Water level:", level)

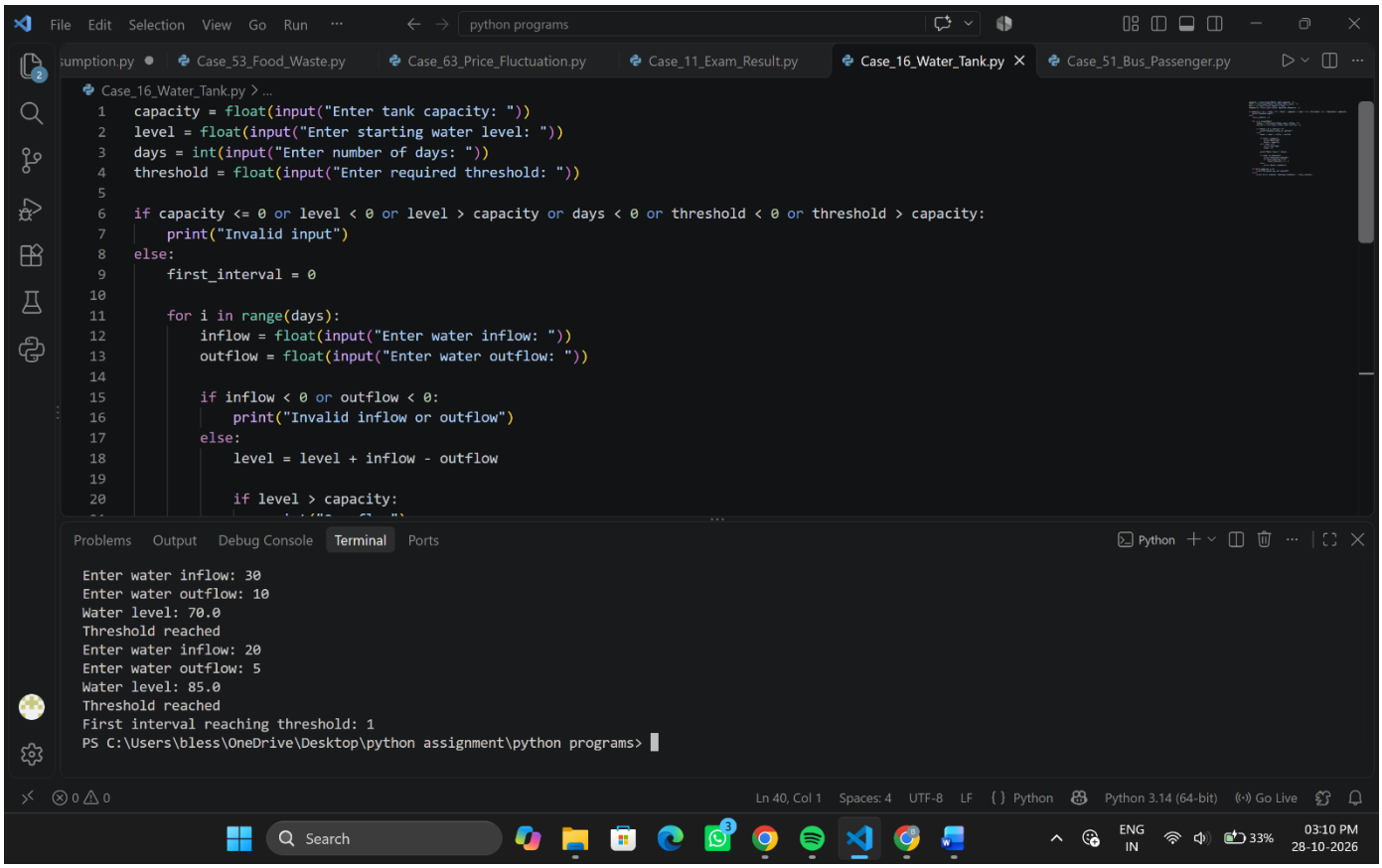
            if level >= threshold:
                print("Threshold reached")
                if first_interval == 0:
                    first_interval = i + 1
            else:
                print("Below threshold")

    if first_interval == 0:
        print("Threshold was not reached")
    else:
        print("First interval reaching threshold:", first_interval)
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Capacity=100, start=50, 2 days; threshold=70; inflow/outflow=(30,10),(20,5)	Levels 70 and 85; first interval = 1	Pass if values match – Pass
TC2	Capacity=100, start=90, threshold=95; inflow=20, outflow=0	Overflow reported; level capped at 100; first interval=1	Pass if values match – Pass
TC3	Capacity=100, start=10, threshold=70; inflow=0, outflow=20	Shortage reported; level set to 0; threshold not reached	Pass if values match – Pass
TC4	Capacity = 0	Invalid input	Pass if it matches -- Pass

Output Screenshot Evidence



Conclusion

The program models changing water levels using loops and conditional checks. It also detects overflow and shortage conditions and identifies the first interval in which the required threshold is reached.

Case 18: Fuel Consumption and Route Comparison

Problem Statement

Calculate fuel requirements and journey costs for several vehicles using distance, mileage, fuel prices, and reserve requirements; identify the lowest cost feasible option.

Objective

To calculate fuel expense for each vehicle and identify the least expensive option.

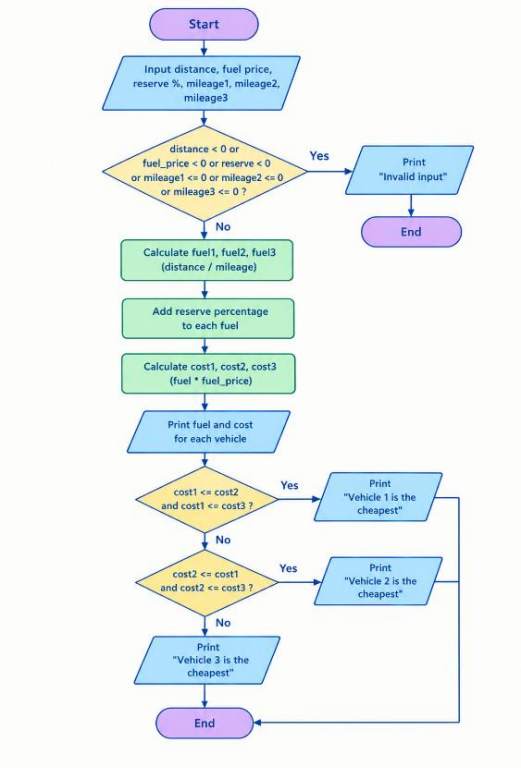
Assumptions

1. Distance and fuel price must be non-negative.
2. Each vehicle's mileage must be positive and expressed as distance per litre.
3. Fuel cost is calculated as distance / mileage × fuel price.
4. Reserve percentage is non-negative and is added to the required fuel before calculating the journey cost.

Algorithm

1. Start.
2. Read journey distance, fuel price per litre, and reserve percentage.
3. Read the mileage of three vehicles.
4. If distance or fuel price is negative, reserve percentage is negative, or any mileage is zero or negative, display an invalid-input message and stop.
5. Calculate fuel required for each vehicle using distance ÷ mileage.
6. Add the required reserve percentage to the fuel required.
7. Calculate the journey cost for each vehicle using fuel with reserve × fuel price.
8. Display the fuel required with reserve and cost for all three vehicles.
9. Compare the costs and display the cheapest vehicle.
10. Stop.

Flowchart



Pseudocode

```
START
INPUT distance, fuel_price, reserve, mileage1, mileage2, mileage3
IF distance < 0 OR fuel_price < 0 OR reserve < 0 OR any mileage ≤ 0 THEN
 OUTPUT Invalid input
ELSE
 fuel1 ← distance / mileage1
 fuel2 ← distance / mileage2
 fuel3 ← distance / mileage3
 fuel1 ← fuel1 + (fuel1 × reserve / 100)
 fuel2 ← fuel2 + (fuel2 × reserve / 100)
 fuel3 ← fuel3 + (fuel3 × reserve / 100)
 cost1 ← fuel1 × fuel_price
 cost2 ← fuel2 × fuel_price
 cost3 ← fuel3 × fuel_price
 OUTPUT all fuel requirements and costs
 IF cost1 ≤ cost2 AND cost1 ≤ cost3 THEN
  OUTPUT Vehicle 1 is cheapest
 ELSE IF cost2 ≤ cost1 AND cost2 ≤ cost3 THEN
  OUTPUT Vehicle 2 is cheapest
 ELSE
  OUTPUT Vehicle 3 is cheapest
STOP
```

Python Source Code

```
distance = float(input("Enter journey distance: "))
fuel_price = float(input("Enter fuel price per litre: "))
reserve = float(input("Enter reserve percentage: "))

mileage1 = float(input("Enter mileage of Vehicle 1: "))
mileage2 = float(input("Enter mileage of Vehicle 2: "))
mileage3 = float(input("Enter mileage of Vehicle 3: "))

if distance < 0 or fuel_price < 0 or reserve < 0 or mileage1 <= 0 or mileage2 <= 0 or mileage3 <= 0:
    print("Invalid input")
else:
    fuel1 = distance / mileage1
    fuel2 = distance / mileage2
    fuel3 = distance / mileage3

    fuel1 = fuel1 + (fuel1 * reserve / 100)
    fuel2 = fuel2 + (fuel2 * reserve / 100)
    fuel3 = fuel3 + (fuel3 * reserve / 100)

    cost1 = fuel1 * fuel_price
    cost2 = fuel2 * fuel_price
    cost3 = fuel3 * fuel_price

    print("Vehicle 1 fuel with reserve:", fuel1)
    print("Vehicle 1 cost:", cost1)

    print("Vehicle 2 fuel with reserve:", fuel2)
    print("Vehicle 2 cost:", cost2)

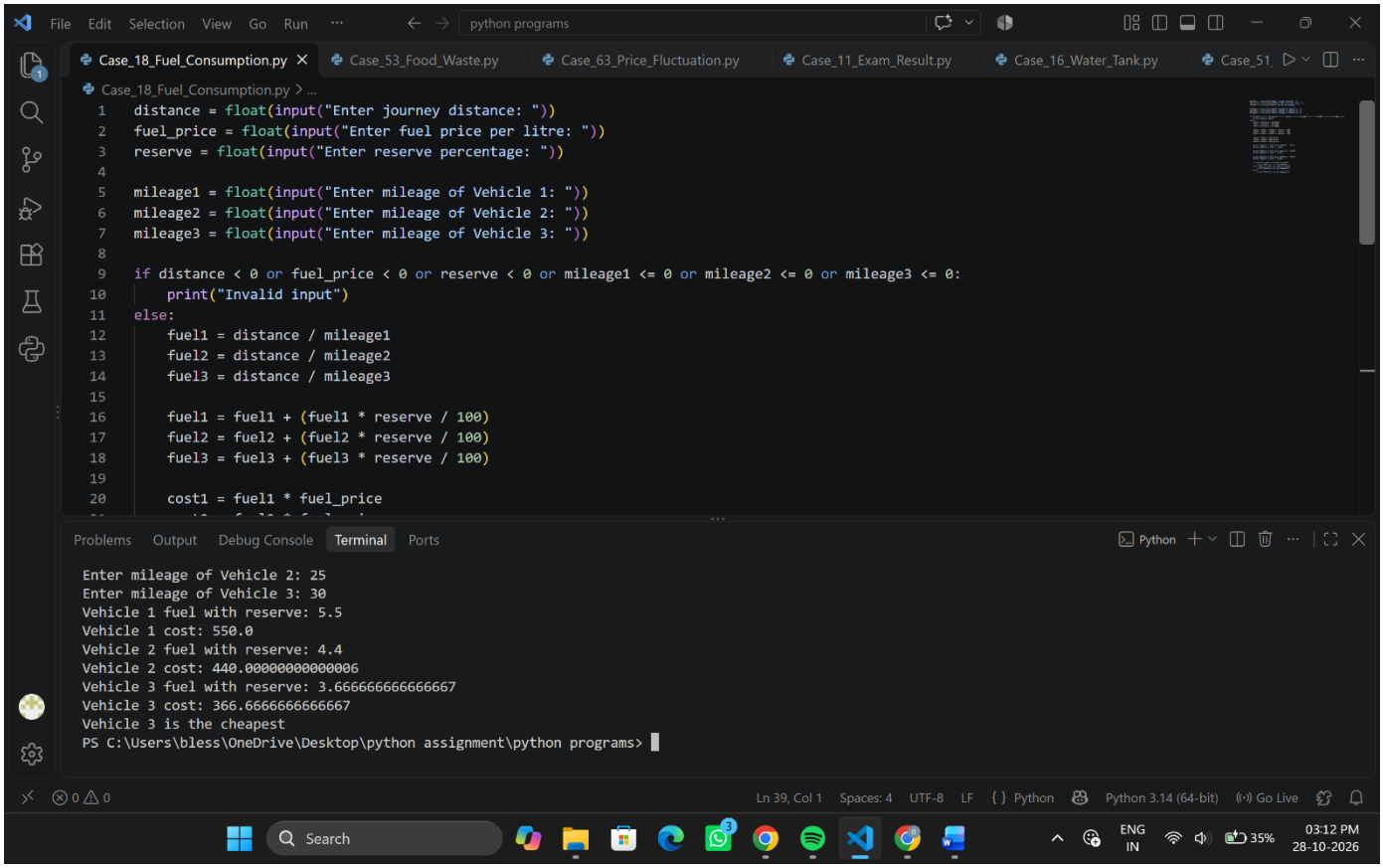
    print("Vehicle 3 fuel with reserve:", fuel3)
    print("Vehicle 3 cost:", cost3)
```

```
    if cost1 <= cost2 and cost1 <= cost3:
        print("Vehicle 1 is the cheapest")
    elif cost2 <= cost1 and cost2 <= cost3:
        print("Vehicle 2 is the cheapest")
    else:
        print("Vehicle 3 is the cheapest")
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Distance=100, price=100, reserve=10%; mileages=20,25,30	Vehicle 1 fuel=5.5 L, cost=550; Vehicle 2 fuel=4.4 L, cost=440; Vehicle 3 fuel=3.67 L, cost=366.67; Vehicle 3 cheapest	Pass if values match – Pass
TC2	Distance=0, positive price, reserve=10%, positive mileages	All fuel requirements and costs=0	Pass if values match – Pass
TC3	One mileage=0	Validation error	Pass if message appears – Pass

Output Screenshot Evidence



Conclusion

The program uses arithmetic calculations, reserve percentage, validation, and cost comparison to evaluate fuel requirements and identify the cheapest vehicle.

Case 51: Bus Passenger Occupancy Simulation

Problem Statement

Process boarding and alighting counts across multiple stops, calculate passenger counts at each stop, and flag any capacity violations.

Objective

To update passenger totals across stops and flag capacity violations.

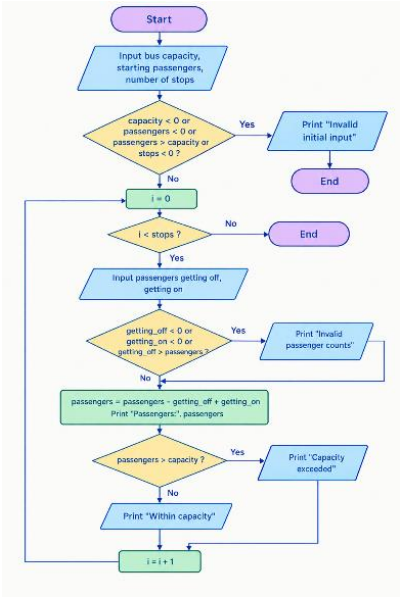
Assumptions

1. Capacity, starting passengers, and number of stops are non-negative integers.
2. Starting passengers cannot exceed capacity.
3. Getting-on and getting-off counts must be non-negative.
4. Passengers getting off cannot exceed the current number of passengers.
5. Capacity is checked after each valid stop.

Algorithm

1. Start.
2. Read capacity, starting passengers, and number of stops.
3. Validate the initial values: capacity and stops must be non-negative, and starting passengers must be between zero and capacity.
4. If initial values are invalid, display Invalid initial input and stop.
5. For each stop, read passengers getting off and getting on.
6. If either count is negative or getting off exceeds current passengers, display Invalid passenger counts and continue to the next stop.
7. Otherwise subtract getting-off passengers and add getting-on passengers.
8. Display the passenger count.
9. If passengers exceed capacity, display Capacity exceeded; otherwise display Within capacity.
10. Stop after all stops.

Flowchart



Pseudocode

```
START
INPUT capacity, passengers, stops
VALIDATE all initial values ≥ 0
FOR each stop:
  INPUT getting_off, getting_on
  IF either count < 0 OR getting_off > passengers THEN OUTPUT Invalid data
  ELSE:
    passengers ← passengers – getting_off + getting_on
    OUTPUT passengers
    IF passengers > capacity THEN OUTPUT Capacity exceeded
    ELSE OUTPUT Within capacity
STOP
```

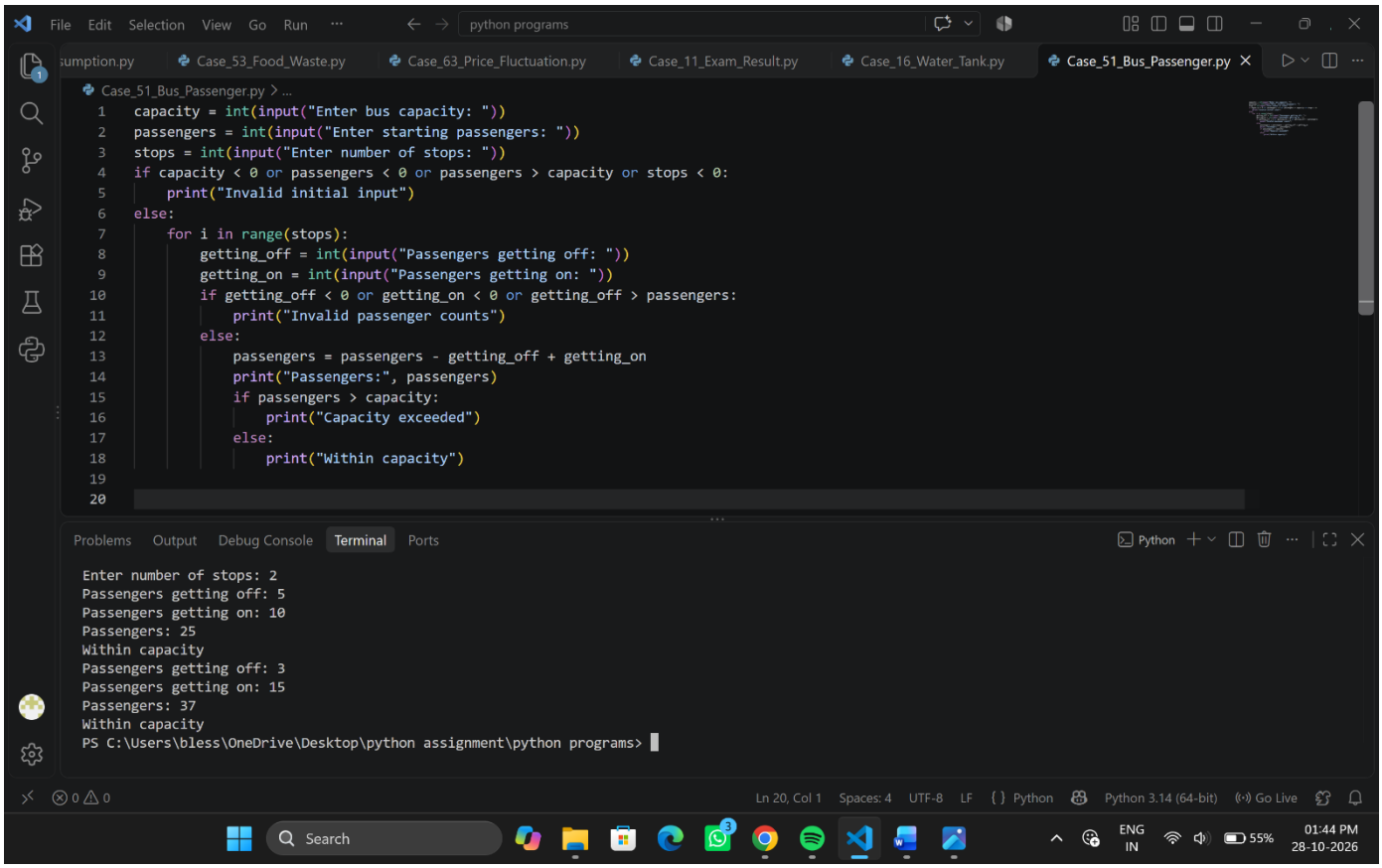
Python Source Code

```
capacity = int(input("Enter bus capacity: "))
passengers = int(input("Enter starting passengers: "))
stops = int(input("Enter number of stops: "))
if capacity < 0 or passengers < 0 or passengers > capacity or stops < 0:
    print("Invalid initial input")
else:
    for i in range(stops):
        getting_off = int(input("Passengers getting off: "))
        getting_on = int(input("Passengers getting on: "))
        if getting_off < 0 or getting_on < 0 or getting_off > passengers:
            print("Invalid passenger counts")
        else:
            passengers = passengers - getting_off + getting_on
            print("Passengers:", passengers)
            if passengers > capacity:
                print("Capacity exceeded")
            else:
                print("Within capacity")
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Capacity=40, start=20; stop1 off=5 on=10; stop2 off=3 on=15	After stop1=25 within capacity; after stop2=37 within capacity	Pass if values match – Pass
TC2	Capacity=30, start=25; off=0, on=10	Passengers=35; capacity exceeded	Pass if values match – Pass
TC3	Start=5; getting off=8	Invalid passenger counts	Pass if message appears – Pass
TC4	Starting passengers greater than capacity	Invalid initial Input	Pass if message appears -- Pass

Output Screenshot Evidence



Conclusion

The program demonstrates loop-based passenger tracking and conditional capacity checking.

Case 53: Food Waste Monitoring System

Problem Statement

Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

Objective

To measure daily leftover meals as a percentage of meals prepared and flag excessive wastage.

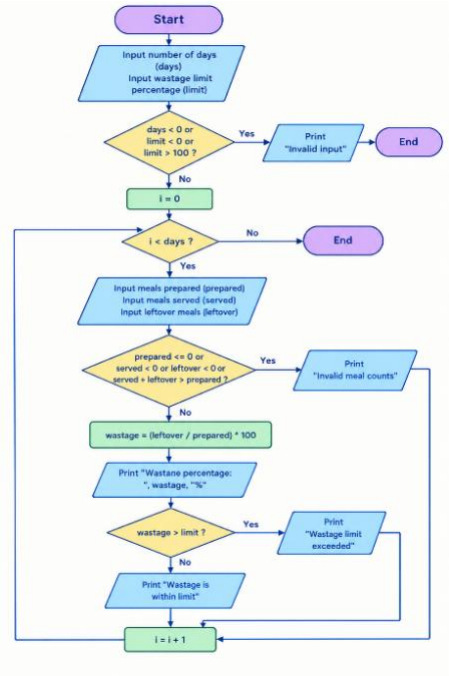
Assumptions

1. Number of days must be a non-negative integer.
2. Wastage limit must be between 0% and 100%.
3. Meals prepared must be greater than zero.
4. Meals served and leftover counts must be non-negative.
5. Meals served plus leftovers cannot exceed meals prepared.

Algorithm

1. Start.
2. Read number of days and wastage limit.
3. If days is negative or the limit is outside 0–100, display Invalid input and stop.
4. For each day, read meals prepared, meals served, and leftover meals.
5. If prepared is not positive, any count is negative, or served plus leftover exceeds prepared, display Invalid meal counts and continue to the next day.
6. Calculate wastage percentage = leftover ÷ prepared × 100.
7. Display the wastage percentage.
8. If wastage is greater than the limit, display Wastage limit exceeded; otherwise display Wastage is within limit.
9. Stop after all days.

Flowchart



Pseudocode

```
START
INPUT days, limit
IF days < 0 OR limit < 0 OR limit > 100 THEN OUTPUT Invalid input
ELSE FOR each day
    INPUT prepared, served, leftover
    IF prepared ≤ 0 OR served < 0 OR leftover < 0 OR served + leftover > prepared THEN OUTPUT Invalid meal counts
    ELSE
        wastage ← leftover / prepared × 100
        OUTPUT wastage
        IF wastage > limit THEN OUTPUT Wastage limit exceeded
        ELSE OUTPUT Wastage is within limit
STOP
```

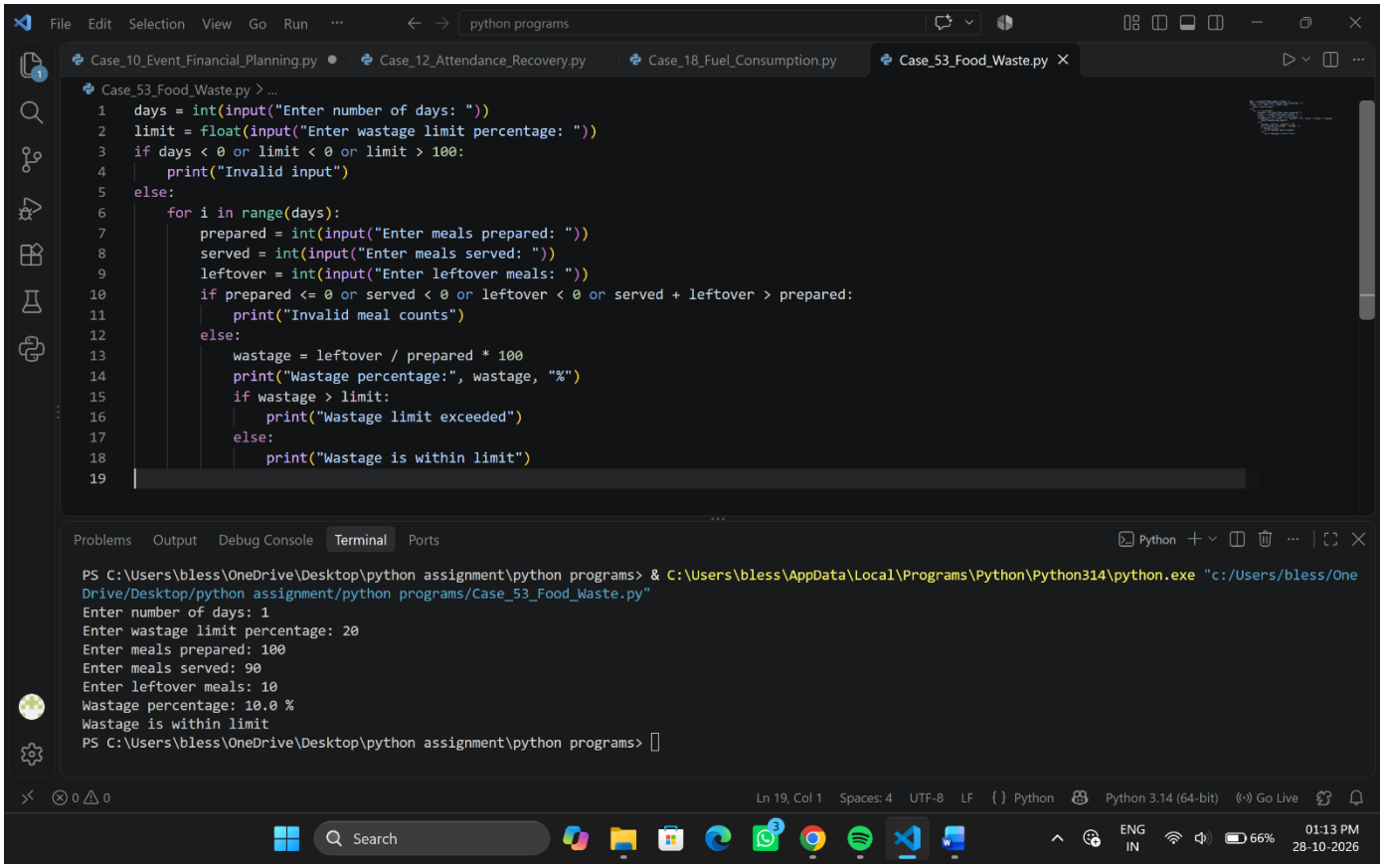
Python Source Code

```
days = int(input("Enter number of days: "))
limit = float(input("Enter wastage limit percentage: "))
if days < 0 or limit < 0 or limit > 100:
    print("Invalid input")
else:
    for i in range(days):
        prepared = int(input("Enter meals prepared: "))
        served = int(input("Enter meals served: "))
        leftover = int(input("Enter leftover meals: "))
        if prepared <= 0 or served < 0 or leftover < 0 or served + leftover > prepared:
            print("Invalid meal counts")
        else:
            wastage = leftover / prepared * 100
            print("Wastage percentage:", wastage, "%")
            if wastage > limit:
                print("Wastage limit exceeded")
            else:
                print("Wastage is within limit")
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Days = 1, Prepared=100, served=90, leftover=10, limit=20%	Wastage=10%; within limit	Pass if values match – Pass
TC2	Days = 1, Prepared=100, served=70, leftover=30, limit=20%	Wastage=30%; limit exceeded	Pass if values match – Pass
TC3	Prepared=100, served = 80, leftover = 30	Invalid meal counts	Pass if message appears – Pass

Output Screenshot Evidence



Conclusion

The program uses loops, percentage calculations, and conditional checks to monitor food wastage.

Case 63: Multi-Product Price Fluctuation Analysis

Problem Statement

Process historical product prices stored in lists, calculate percentage changes, identify maximum fluctuations, and flag products exceeding a supplied threshold.

Objective

To monitor product price changes and identify the product with the greatest absolute percentage change.

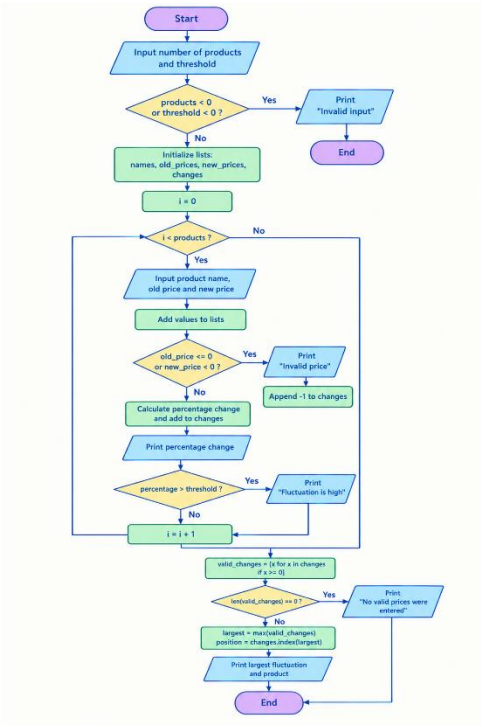
Assumptions

1. Product count and threshold are non-negative.
2. Old price must be greater than zero
3. New price must be non-negative.
4. Absolute percentage change is used, so both increases and decreases are compared by magnitude.
5. If no valid prices are entered, the program reports that no valid prices were entered.

Algorithm

1. Start.
2. Read the number of products and fluctuation threshold.
3. If product count or threshold is negative, display Invalid input and stop.
4. Create lists to store product names, old prices, new prices, and percentage changes.
5. For each product, read its name, old price, and new price and store them in the lists.
6. If old price is zero or negative, or new price is negative, display Invalid price.
7. Otherwise, calculate absolute percentage change and store it in the changes list.
8. Display the percentage change for each valid product.
9. If the percentage change is greater than the threshold, display Fluctuation is high.
10. Find the largest percentage change and corresponding product from the lists.
11. Display the largest fluctuation and product, or display No valid prices were entered if none are valid.
12. Stop.

Flowchart



Pseudocode

```
START
INPUT products, threshold
IF products < 0 OR threshold < 0 THEN
 OUTPUT Invalid input
ELSE
 names ← empty list
 old_prices ← empty list
 new_prices ← empty list
 changes ← empty list
 FOR each product
  INPUT name, old_price, new_price
  ADD name to names
  ADD old_price to old_prices
  ADD new_price to new_prices
  IF old_price ≤ 0 OR new_price < 0 THEN
   OUTPUT Invalid price
   ADD -1 to changes
  ELSE
   percentage ← ABS(new_price – old_price) / old_price × 100
   ADD percentage to changes
   OUTPUT percentage
   IF percentage > threshold THEN
    OUTPUT Fluctuation is high
 IF there is no valid change THEN
  OUTPUT No valid prices were entered
 ELSE
  largest ← maximum valid value in changes
  position ← position of largest in changes
  OUTPUT largest and names[position]
STOP
```

Python Source Code

```
products = int(input("Enter number of products: "))
threshold = float(input("Enter fluctuation threshold: "))

if products < 0 or threshold < 0:
    print("Invalid input")
else:
    names = []
    old_prices = []
    new_prices = []
    changes = []

    for i in range(products):
        name = input("Enter product name: ")
        old_price = float(input("Enter old price: "))
        new_price = float(input("Enter new price: "))

        names.append(name)
        old_prices.append(old_price)
        new_prices.append(new_price)

        if old_price <= 0 or new_price < 0:
            print("Invalid price")
            changes.append(-1)
        else:
```

```
            percentage = abs(new_price - old_price) / old_price * 100
            changes.append(percentage)

            print("Percentage change:", percentage, "%")

            if percentage > threshold:
                print("Fluctuation is high")

    valid_changes = [x for x in changes if x >= 0]

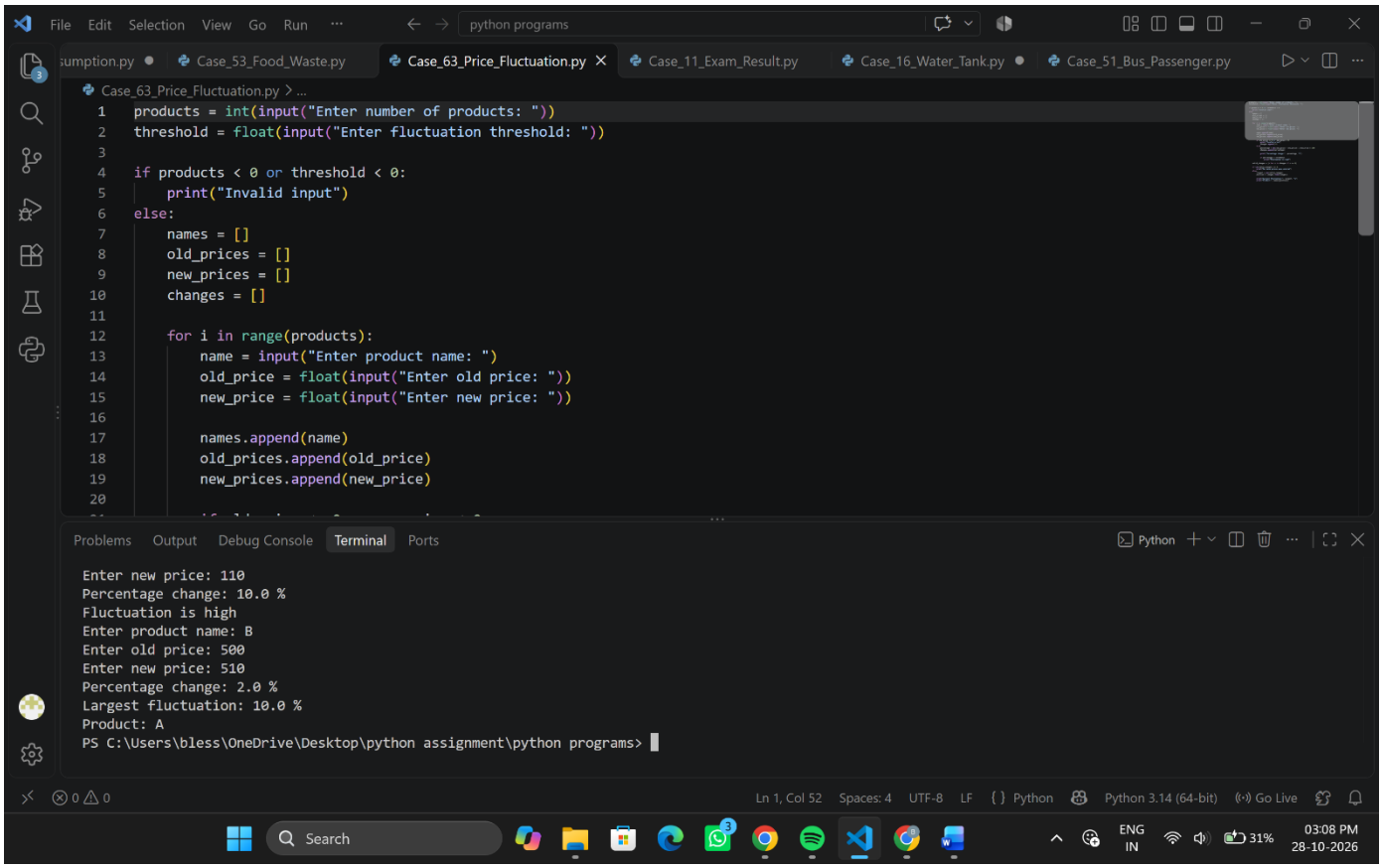
    if len(valid_changes) == 0:
        print("No valid prices were entered")
    else:
        largest = max(valid_changes)
        position = changes.index(largest)

        print("Largest fluctuation:", largest, "%")
        print("Product:", names[position])
```

Test Cases

Test ID	Input / condition	Expected output	Pass criterion
TC1	Products = 2, threshold = 5%; A old = 100 new = 110; B old = 500 new = 510	A=10% high; B=2%; largest is A at 10%	Pass if values match – Pass
TC2	One product; threshold = 5%; old = 100, new = 100	Change=0%; largest is 0%	Pass if values match – Pass
TC3	Old price=0	Invalid price	Pass if message appears – Pass

Output Screenshot Evidence



Conclusion

The program uses lists, percentage calculations, conditional alerts, and maximum-value tracking to analyse product price fluctuations.

Overall Conclusion

The case studies demonstrate how Python programming can be used to model practical situations. The documented algorithms, flow representations, pseudocode, source code, test cases, and output evidence are intended to support review, testing, and repository submission.